

Research Statement

Siddhartha Prasad

Generative AI is changing who programs and why. Even people who do not identify as programmers are no longer limited to only consuming computational artifacts. They can produce the bespoke code, automations, and even proofs they need, when they need them. This is often described as magically automating the task of programming: natural language goes in, a formal artifact comes out.

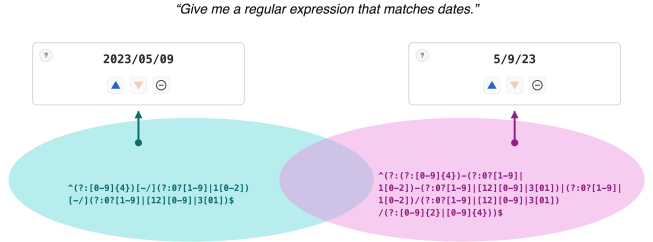
This, however, is an overly mechanical view of what a programmer does. My time in industry as a software engineer taught me that producing code is only one part of the practice of programming. Another subtle but critical demand is the cognitive work of formalizing intent and exercising judgment. Whether you are a professional programmer or someone newly empowered by GenAI, you still have to decide what the system should do, evaluate tradeoffs, and determine whether the result is fit for purpose.

Programming languages (PL) and formal methods (FM) researchers have spent decades developing powerful ways to relate specifications of behavior to program implementations. Yet even experienced programmers use only a thin slice of this toolkit in everyday work. Formal tools typically treat specifications as settled inputs, rather than things programmers discover and revise through the process of building software.

My research repurposes **formal machinery traditionally aimed at automated analysis** to **support human understanding**. I ask how PL and FM techniques can help programmers discover, test, and revise their intent as they work, rather than entering only after programmers have had to work through these hard decisions unaided. Three of my projects illustrate this approach, each through a question programmers face.

What should my system do? Programmers increasingly rely on GenAI to translate informal requirements into logical formulae, security policies, and other formal specifications. This is convenient but risky: natural language is ambiguous, relevant domain knowledge is often unstated, and the same prompt can yield a different yet plausible output each time it is run.

PICK [1] is an algorithm that exploits this variability to help users build confidence in generated specifications. Rather than ask a model for a single answer, it collects several candidates and guides the programmer through an interactive process to determine which, if any, captures their intended behavior. For that interaction to be productive, it must ask programmers questions they can understand, and not ask too many of them.

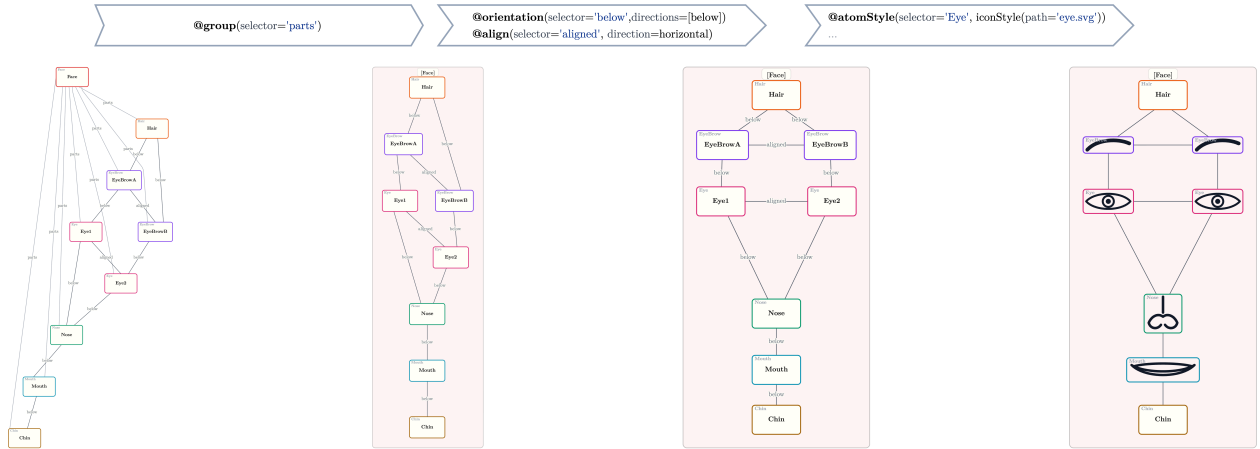


PICK's design rests on two observations. First, many of these specification languages enjoy **closure properties**, or meaningful approximations thereof. This lets PICK compute the semantic differences between candidates and sample from them, so every scenario it shows is one on which plausible interpretations actually disagree. The questions stay few, and each one is decisive. Second, cognitive science has long demonstrated the **value of concrete examples** for reasoning about abstract rules [2]. PICK therefore asks programmers to judge sampled scenarios that witness behavioral differences between candidate specifications, rather than only compare abstract formulae. Doing so puts their understanding to the test, surfacing gaps that can remain hidden at the abstract level [3]. It is far easier to decide whether a specific string should be accepted by your parser than to compare two complex grammars.

I have instantiated PICK in working systems for three specification domains: **regular expressions**², access control policies, and **Linear Temporal Logic**² (LTL). User studies showed that PICK users identified the correct candidate, or recognized that none was correct, more accurately than those choosing among candidates directly. Even more strikingly, novices using PICK for LTL performed as well as experts not using the tool.

What am I looking at? Programming practice depends heavily on the ability to inspect the values programs produce, yet the basic tools for doing so (print statements, REPLs, debuggers) are overwhelmingly textual. But for many program values, understanding depends not only on seeing their parts, but also on seeing how those parts relate. Decades of cognitive science research shows that **diagrams can make structure much easier to perceive** [4–6]. So why not generate a diagram whenever inspection calls for one? Because getting one is expensive, and modifying it can be even more so. Before anything appears on screen, the programmer must interrupt their flow to make representational choices (what to show, how values map to diagram elements) and then implement them through low-level drawing code (computing coordinates, managing layout and rendering). Only a small part of that code captures the structure the programmer actually cares about; the rest is scaffolding. Worse, the resulting diagram may reveal that different parts or relationships matter, sending the programmer back through the same process to construct a different view. The workflow demands commitment precisely where inspection demands flexibility. To serve inspection, diagrams must be cheap to obtain and cheap to revise.

Spytial [7, 8] is a language for diagram-guided inspection built around the observation that structured values already have an implicit graph structure. Their constituent values become nodes, and relationships among them become edges. That graph determines what there is to draw, so **programmers can get a complete structural view** before writing any diagramming rules. This turns diagramming into a **specification problem over how those elements should be laid out**.



Informed by cognitive science research [5, 6, 9, 10], the language’s vocabulary lets programmers constrain **spatial relationships between diagram elements** rather than specify explicit coordinates. **A constraint solver fills in the rest**. The value remains the source of truth, so information cannot disappear unless explicitly hidden. Rules refine a shared space of possible layouts, so they compose without ordering effects. Spytial is also designed for the cases inspection is meant to uncover: when the value violates the diagrammer’s assumptions. This is when programmers need a diagram most! Thus, instead of only reporting a textual error when requirements conflict, Spytial also produces a *counterfactual diagram* that **relaxes the conflict, showing what went wrong in the context of the rest of the value**.

I developed Spytial’s metatheory and proved key properties of its declarative semantics, including refinement and order-independent composition. The same semantic core supports a range of programming settings. I have integrated Spytial with Python[℄], Rust[℄], Pyret[℄], and Lean[℄], and its performance is fast enough for interactive use [11] in all four. In user studies, Spytial diagrams improved comprehension over generic graph layouts, and its counterfactual conflict reports outperformed textual ones.

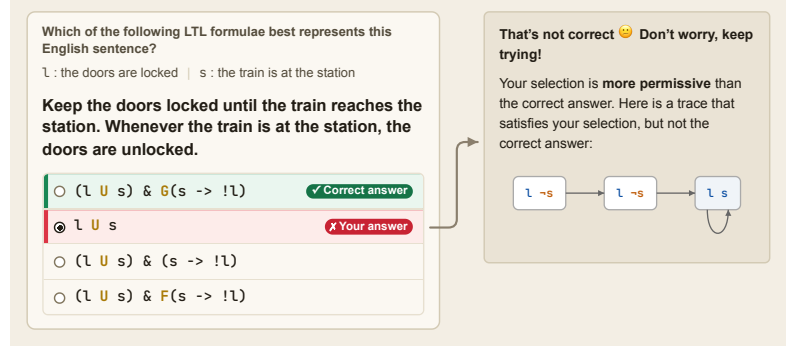
Does this formula mean what I think it means? Linear Temporal Logic (LTL) is used to specify the behavior of systems in domains such as robotics and autonomous systems. An engineer designing a train system might therefore quite reasonably encode “*keep the doors locked until the train reaches the station*” as $1 \text{ UNTIL } s$. Unfortunately, this reflects a common misconception about LTL. UNTIL requires s eventually to occur, but does not require 1 to stop holding once it does. The specification therefore permits the train

to sit at the station with its passengers still locked inside.

As part of a broader project on improving LTL education, I used **grounded theory methods** to identify several such recurring misconceptions in how users reason about the logic [12]. Each corresponds to a coherent **alternative interpretation of LTL’s semantics**.

Educators are well positioned to address these misconceptions, but they cannot rebuild a curriculum around every new empirical finding or provide each learner with individualized diagnosis and remediation. Therefore, any intervention has to support existing instruction, generate its own exercises, and adapt them to the specific misconceptions that an individual learner exhibits.

The **LTL Tutor**² [13] is a misconception-first adaptive tutor designed to complement existing instruction with little disruption. It tailors instruction to a learner’s mental model of the logic’s semantics. Its design rests on my insight that **semantic misconceptions** can be expressed through **syntactic mutation**. Mutation rules rewrite a formula into one that, under the semantics entailed by a particular misconception, behaves as the original does under LTL’s actual semantics. These mutants serve as distractors in automatically generated multiple-choice questions. When chosen, they reveal how the learner is interpreting the logic. The Tutor can then use **distinguishing traces** to explain the difference and **generate subsequent exercises targeting that misconception**.



To date, over 600 learners have used the LTL Tutor to answer over 6,000 questions. Analysis of these responses shows that learners improve with continued use, answering more accurately and showing corresponding improvements in their misconception profiles.

Future Work

The programming practice many of us learned centered on the task of implementation. Writing code put programmers’ understanding of what they were building to the test. Implementing an idea exposed questions no one had thought to ask and choices no one had realized they needed to make. Sometimes the answer lay in an unstated requirement; other times, the programmer had to make a judgment call.

The programming practice now emerging shifts the center of gravity from implementation toward articulating and refining intent. Not only does GenAI let experienced programmers spend less time producing implementations and more time specifying, inspecting, and revising them; it also moves us closer to the long-standing vision of end-user programming, making software creation practical for social scientists and other domain experts [14]. No matter their background, they will still need to understand what their systems do, work out what they *should* do, and discover when their own understanding is flawed [15]. As users do less of the implementation themselves, our tools should not lull them into passive acceptance of whatever is produced on their behalf. My research vision is to make the cognitive work that was once an incidental and uneven byproduct of writing code an explicit object of language and tool design, including for end users who enter programming through GenAI. Below, I develop three directions within this broader agenda.

Specifications that Evolve through Interaction Autoformalization’s allure lies in its promise to alleviate the apparent burden of human judgment: natural language goes in, the right formal artifact comes out. In my experience, it just pushes that judgment downstream, where failures are harder to see and more expensive to fix. I instead plan to use this machinery to generate *drafts*. As with writing, it is easier to critique or revise a draft than to start from scratch.

Once users have a draft, formal techniques can support that critique and revision. My previous work on

PICK offers evidence that examples can be used to ask *Is this behavior right?*. But examples can drive much richer questions. A **solver can expose behaviors that distinguish** the current draft from a proposed revision, or counterexamples to properties the user expects the draft to satisfy. Existing PL and FM techniques typically select examples to optimize algorithmic refinement loops, but the criteria for a useful example change when the audience is human. **The right example depends on the critique it should support**. Cognitive science offers principles for making that choice: contrasting cases can highlight important distinctions [16–18], comparison across examples can reveal shared structure [19], and feature highlighting can connect observed behavior to the aspects that explain it [20]. I aim to build interactive systems that elicit critique and support revision where it matters, rather than chase the illusion of one-shot specification generation.

Interrogable Interfaces from Formal Representations Computational notebooks transformed scientific communication by making computation part of the document itself. Readers can change an input, rerun the computation, and ask: *What happens now?* But the results those computations produce (a chart, a query result, an optimal schedule) are still largely read-only. This is a waste: the formal machinery behind each result lives *in the notebook*, and it can answer questions [21] the static output cannot. Notebooks made computations live; I plan to make the artifacts they produce live as well, by turning their underlying formal representations into interrogable interfaces.

The design of these interfaces must be informed by the questions users might want to ask. Some concern the result’s status among its alternatives: *Must this node be to the left of that one?* *Is this schedule the only optimum?* An interface for modal queries over the space of acceptable results can answer these. Other questions ask why: *Which requirements force this feature?* *What would change if they were relaxed?* Here the machinery can identify the requirements responsible or generate a counterfactual that answers the what-if. Errors and failures should be interrogable too. When requirements conflict, **solvers can often identify an unsatisfiable core**: a small subset of constraints that cannot all be satisfied. But a core is descriptive, not explanatory. It localizes the failure by naming the constraints involved, while leaving the user to reconstruct what that failure means for the object they are trying to understand.

My previous work on counterfactual diagrams in Sptyal shows another possibility. Rather than only name the conflicting constraints, the failure report shows what becomes possible when the conflict is relaxed, making the **consequences of those constraints visible** in the context of the object being inspected. By turning formal representations into interrogable interfaces, I aim to let users reason not only from computational results, but through the machinery that produced them.

Diagrams for Agentic Reasoning Humans use diagrams not only to present information, but also to communicate and reason: spatial organization can make relationships easier to find than repeatedly traversing a sentential description. Can diagrams provide a shared medium for humans and agents to communicate and reason? If so, what representation of a diagram best supports that use? Vision-language models make it increasingly easy to include diagrams in agent workflows, but rendered images are only one possible representation. A diagram might be shared as a rendered image, an SVG, an abstract graph (e.g., DOT) or be enriched with spatial requirements (e.g., Sptyal). By comparing these representations on collaborative tasks, I aim to understand whether diagrams improve human-agent reasoning, which aspects of diagrammatic structure matter, and which representations make those aspects available to agents.

Conclusion GenAI is shifting the most important part of a programmer’s job from producing code to specifying what a system should do. I want to empower everyone, from end-user programmers to professional software engineers, in this evolving world by bringing the **powerful machinery of PL and FM** to bear on the **critical cognitive tasks that programming still demands**. Informed by insights from cognitive science and human factors, I want to develop techniques and tools that help programmers understand what their systems do, determine what they *should* do, and get answers to important semantic questions like what-if, what-else, and why.

References

- [1] Siddhartha Prasad, Skyler Austen, Kathi Fisler, and Shriram Krishnamurthi. “Meaningful Human-in-the-Loop Checking of GenAI Synthesis for Restricted Languages”. In: *ECOOP*. 2026. DOI: [10.4230/LIPICs.ECOOP.2026.22](https://doi.org/10.4230/LIPICs.ECOOP.2026.22).
- [2] R. K. Atkinson, S. J. Derry, A. Renkl, and D. Wortham. “Learning from Examples: Instructional Principles from the Worked Examples Research”. In: *Review of Educational Research* 70.2 (2000), pp. 181–214. DOI: [10.3102/00346543070002181](https://doi.org/10.3102/00346543070002181).
- [3] Leonid Rozenblit and Frank Keil. “The misunderstood limits of folk science: An illusion of explanatory depth”. In: *Cognitive Science* (2002).
- [4] Barbara Tversky and Masaki Suwa. “Thinking with Sketches”. In: *Tools for Innovation*. 2009. ISBN: 978-0-19-538163-4. DOI: [10.1093/acprof:oso/9780195381634.003.0004](https://doi.org/10.1093/acprof:oso/9780195381634.003.0004).
- [5] Jill H Larkin and Herbert A Simon. “Why a Diagram is (Sometimes) Worth Ten Thousand Words”. In: *Cognitive Science* (1987). DOI: [10.1111/j.1551-6708.1987.tb00863.x](https://doi.org/10.1111/j.1551-6708.1987.tb00863.x).
- [6] Barbara Tversky. “Spatial Schemas in Depictions”. In: *Spatial Schemas and Abstract Thought*. 2001. ISBN: 978-0-262-31616-3. DOI: [10.7551/mitpress/6392.003.0006](https://doi.org/10.7551/mitpress/6392.003.0006).
- [7] Siddhartha Prasad, Michael Tu, Karan Kashyap, Tim Nelson, and Shriram Krishnamurthi. “Diagramming Program Values by Spatial Refinement”. In: *PLDI* (2026). DOI: [10.1145/3808275](https://doi.org/10.1145/3808275). URL: <https://doi.org/10.1145/3808275>.
- [8] Siddhartha Prasad, Ben Greenman, Tim Nelson, and Shriram Krishnamurthi. “Lightweight Diagramming for Lightweight Formal Methods: A Grounded Language Design”. In: *ECOOP*. 2025. ISBN: 978-3-95977-373-7. DOI: [10.4230/LIPICs.ECOOP.2025.26](https://doi.org/10.4230/LIPICs.ECOOP.2025.26).
- [9] Kurt Koffka. “Perception: An Introduction to the Gestalt Theory”. In: *Psychological Bulletin* 19 (1922), pp. 531–585.
- [10] Robert Goldstone. “An efficient method for obtaining similarity data”. In: *Behavior Research Methods, Instruments, & Computers* (1994). DOI: [10.3758/BF03204653](https://doi.org/10.3758/BF03204653).
- [11] Jakob Nielsen. *Usability Engineering*. 1994. ISBN: 9780080520292.
- [12] Ben Greenman, Siddhartha Prasad, Antonio Di Stasio, Shufang Zhu, Giuseppe De Giacomo, Shriram Krishnamurthi, Marco Montali, Tim Nelson, and Milda Zizyte. “Misconceptions in Finite-Trace and Infinite-Trace Linear Temporal Logic”. In: *FM 2024*. 2024. DOI: [10.1007/978-3-031-71162-6_30](https://doi.org/10.1007/978-3-031-71162-6_30).
- [13] Siddhartha Prasad, Ben Greenman, Tim Nelson, and Shriram Krishnamurthi. “A Misconception-Driven Adaptive Tutor for Linear Temporal Logic”. In: *CAV 2025*. 2025. DOI: [10.1007/978-3-031-98685-7_9](https://doi.org/10.1007/978-3-031-98685-7_9).
- [14] Amy J. Ko, Robin Abraham, Laura Beckwith, Alan Blackwell, Margaret Burnett, Martin Erwig, Chris Scaffidi, Joseph Lawrance, Henry Lieberman, Brad Myers, Mary Beth Rosson, Gregg Rothermel, Mary Shaw, and Susan Wiedenbeck. “The State of the Art in End-User Software Engineering”. In: *ACM Comput. Surv.* (Apr. 2011). DOI: [10.1145/1922649.1922658](https://doi.org/10.1145/1922649.1922658).
- [15] Teppo Felin, Mari Sako, and Jessica Hullman. “AI and Actor-Specific Decisions”. In: *AI & SOCIETY* (2026). DOI: [10.1007/s00146-026-03202-7](https://doi.org/10.1007/s00146-026-03202-7).
- [16] James J Gibson and Eleanor J Gibson. “Perceptual learning: Differentiation or enrichment?” In: *Psychological review* 62.1 (1955). Publisher: American Psychological Association, pp. 32–41. DOI: [10.1037/h0048826](https://doi.org/10.1037/h0048826).
- [17] John D. Bransford, Jeffery J. Franks, Nancy J. Vye, and Robert D. Sherwood. “New Approaches to Instruction: Because Wisdom Can’t Be Told.” In: *Similarity and Analogical Reasoning*. 1989. DOI: [10.1017/CB09780511529863.022](https://doi.org/10.1017/CB09780511529863.022).
- [18] Daniel L. Schwartz, Catherine C. Chase, Marily A. Oppezzo, and Doris B. Chin. “Practicing Versus Inventing With Contrasting Cases: The Effects of Telling First on Learning and Transfer”. In: *Journal of Educational Psychology* (2011). DOI: [10.1037/a0025140](https://doi.org/10.1037/a0025140).
- [19] Dedre Gentner, Jeffrey Loewenstein, and Leigh Thompson. “Learning and Transfer: A General Role for Analogical Encoding”. In: *Journal of Educational Psychology* (2003). DOI: [10.1037/0022-0663.95.2.393](https://doi.org/10.1037/0022-0663.95.2.393).
- [20] Toshiya Miyatsu, Reshma Gouravajhala, Robert M. Nosofsky, and Mark A. McDaniel. “Feature Highlighting Enhances Learning of a Complex Natural-Science Category”. In: *Journal of Experimental Psychology: Learning, Memory, and Cognition* (2019). DOI: [10.1037/xlm0000538](https://doi.org/10.1037/xlm0000538).
- [21] Amy J. Ko and Brad A. Myers. “Designing the whyline: a debugging interface for asking questions about program behavior”. In: *CHI ’04*. 2004. DOI: [10.1145/985692.985712](https://doi.org/10.1145/985692.985712).